rc-aarch

把 rCore 移植到 AArch64，跑进 Jailhouse 隔离分区

软件测试课堂展示

树莓派 4B · openEuler MCS · Jailhouse cell3

项目介绍

从教材里的 RISC-V 操作系统，到真实板子上的隔离分区

用 AArch64 重写 rCore-Tutorial 的 9 章迷你操作系统，并在树莓派 4B 的 Jailhouse 非根 cell 里跑通。

从教材到真机

rCore 是 RISC-V 的教材实现；本项目逐章移植到 AArch64，代码、文档、验收 app 一一对应。

隔离环境

运行在树莓派 4B 的 Jailhouse 非根 cell 中，与主系统 Linux 共存，互不影响。

可测、可演示

每章都有验收 app 和日志；一条命令跑完 ch1–ch9，结果自动给出 PASS/FAIL。

规模：内核约 6 千行 Rust； 9 个章节、 17 个用户态测试程序； 11 项逐章验收。

系统能力全景

9 个章节 → 6 项能力；本页只做全景，后面深挖 1 项、用测试数据覆盖其余

启动与异常（ch1–ch2）

裸机入口、异常向量表、系统调用、 EL0 用户态

进程与地址空间（ch3–ch5）

任务切换与抢占、每任务页表、 fork / exec / waitpid

文件系统（ch6）

easy-fs 、 RAM 磁盘、 open / read / write 、块缓存

IPC 与信号（ch7）

管道 pipe / dup 、阻塞读写、信号、 exec 传参

线程与同步（ch8）

线程创建与等待、 mutex 、 semaphore 、 condvar

跨 cell 共享内存（ch7+）

共享内存字节环、 stdin / stdout 、交互 shell

9 章里大部分是“内核机制”，很难在 10 分钟里逐章讲；**所以用逐章测试的结果来证明它们都在，只把共享内存控制台展开讲。**

功能深挖：共享内存控制台

在 ssh 里敲命令，让隔离分区里的 shell 执行并回传输出

问题：板子的串口被主系统占用，虚拟机没有输入通道，只能输出。

方案：主系统侧一个“传声筒”守护进程，和虚拟机通过共享内存通信：一块两边都能读写的内存，加一个字节环（写指针 / 读指针 / 代数）。

效果：在 ssh 里敲命令 → 送进虚拟机 → 它的 shell 执行 → 输出原路回传。

边界：这不是网络协议栈，只是共享内存上的读写约定；主系统用 uio 访问，虚拟机侧就是普通内存读写。



测试体系

真机、自动化、可追溯、可回归

真机测试：所有用例都在树莓派 + Jailhouse 上跑，不用模拟器 —— 避免“模拟器上过、真机不过”。

自动化：一条命令 ./scripts/verify-chapters.sh：启动虚拟机，按 ch1→ch9 逐项断言，输出 PASS/FAIL 表。

可追溯：控制台日志 + 守护进程日志都落盘；失败能定位到具体章节和具体 app。

回归：每次改动重跑全部章节（含旧章节）；本次 11 项断言全部通过。

测试用例（代表）

挑 4 个有数字、可复核的用例；其余见验收总表

用例	验证点	预期 / 实际	结果
pipe_large_test	32 字节管道传 3000 字节；写满阻塞、读空唤醒	校验和 382468	PASS
mutex_test	4 个线程各 1000 次加锁自增	计数 = 4000	PASS
fs_perf + 缓存统计	200 次 open + read ；块缓存命中率	命中率 83%	PASS
03sbrk	sbrk 扩页、跨页读写、非法缩回失败	全部符合预期	PASS

另有用例覆盖：信号处理与屏蔽、 fork/exec/waitpid 、 argv 传参、文件读写、线程创建与等待、块缓存淘汰。

验收结果： 11 / 11 通过

在真机上重新跑一遍后的结果（脚本自动断言）

章节	结果	主要证据
ch1 裸机入口 + UART	PASS	0xfa000000 启动、 MMU 恒等映射、控制台输出完整
ch2 trap / syscall / EL0	PASS	EL0 打印 hello 、 svc 陷入路径正常
ch3 任务切换 + 抢占	PASS	两个 app 交错输出、定时器 tick 来自 EL0
ch4 地址空间 + sbrk	PASS	remap_test 、扩页与跨页读写通过
ch5 fork / exec / waitpid	PASS	forktest 全链路、子进程回收
ch6 文件系统	PASS	filetest 、 cat 读到 hello.txt
ch7 pipe + 信号 + argv	PASS	校验和 382468 、 sig_test 、 argv 正确
ch7+ 共享内存控制台	PASS	shell 交互会话完整（ stdin/stdout ）
ch8 线程 + 同步	PASS	thread / mutex / sync 三个 app 全过
ch9 块缓存	PASS	fs_perf 跑通、命中率 83%
全局	PASS	无 panic 、 initproc 正常退出、系统停机

测试发现的典型缺陷

4 个“看起来能跑、其实悄悄错”的问题

现象	根因	修复
同一地址换了程序，偶发执行旧指令	AArch64 改地址空间后没有失效 I-cache	切换页表时执行 ic iallu
管道数据整片错，父子校验和却相同（假通过）	内核用 SIMD 把用户寄存器现场写坏	TrapContext 保存 / 恢复 q0–q31
新 shell 一启动就退出、命令错乱	共享内存残留输入；读下标写错段	启动清环；下标写自己段
ch8 启动即 panic “进程已回收”	进程控制块的强引用提前归零	创建返回前先登记

教训：只比较“两边各自算出的摘要”，当两边被同一个 bug 写坏时会假通过；跨特权级、跨分区的用例要直接比内容。

扩展性与集成性

不是一个封闭的演示，而是一套可以继续长的基础

扩展性

- 章节可插拔： ch1–ch9 各自分支与文档，新增一章不影响旧章节
- 通道可扩展：共享内存除了 stdin/stdout ，还能承载日志、数据采集、控制命令
- 块设备可替换： RAM 盘 → SD 卡 / eMMC ，接口不变（ BlockDevice ）
- 继续加：更多线程原语、更多 syscall 、多核

集成性

- 隔离性：虚拟机里的故障（ panic 、死循环）不影响板子的主系统
- 与主系统共存：同一块板子、同一套网络和 ssh ，随用随起
- 通用通道：主系统用 uio 访问共享内存，虚拟机侧就是普通内存读写
- 演示友好：一条命令部署，一条命令验收

总结

9 章全部跑通：从裸机 hello 到线程、同步、块缓存，都在真机上验证。

测试驱动：逐章验收脚本， 11 项断言全过；测试过程中发现并修掉 4 类典型缺陷。

可复现：仓库含 11 篇对照文档；部署和验收各一条命令。

仓库： github.com/QewbieJaw/rc-aarch ｜ atomgit.com/EraFei/rc-aarch
文档： docs/01 – docs/11 （逐章对照 + 逐章验收）

谢谢，欢迎提问。